



ADOBE UNIVERSITY HACKATHON 2026

Round 3 — Build the Agent Skill Marketplace

Builds on Round 2

Round 2 tested reasoning on paper: why a brand is invisible / stale / bouncing in AI apps, and what to do about it. Round 3 tests whether you can encode that reasoning into reusable agent skills, so that a general AI agent, pointed at any website, can audit it automatically — detecting the problems that hurt both its AI discoverability (getting found and cited) and its on-site engagement (keeping the visitor once they arrive) — and produce a report of findings plus suggested actions to fix them.

How to approach it — learn from the wild

We won't hand you a list of test sites. Part of the challenge is field research: go find real websites that AI assistants cite well versus ones they ignore or misrepresent, and work out what makes the difference. Distil those learnings — the concrete, repeatable signals that separate cited from uncited (the Round-2 failure modes) — into your skill, each with evidence and a severity.

We won't ask which sites you studied, and none of them are used in grading. Your skill is judged only on how well it generalizes to sites you've never seen (see "How submissions are evaluated" below).

Design for patterns, not fit-to-examples. You will be evaluated on unseen websites — focus on discovering repeatable root causes, not memorizing specific site characteristics.

1. What you submit

A single Agent Skill Marketplace: a package containing one or more skills, each authored in the standard Agent Skills (agentskills.io) format (a SKILL.md with YAML frontmatter + instructions, optional bundled scripts/ and references/), tied together by a marketplace manifest and exactly one designated endpoint skill that receives the audit request and emits the final report. Decomposing your reasoning into multiple focused skills (one per concern) is the point of the marketplace format, and is rewarded in the rubric below. A marketplace with only one skill is accepted as a floor — it's a valid submission, not the target.

Given a website, the marketplace's endpoint skill audits it and produces an audit report containing:

- Problems found — the issues hurting the brand's AI discoverability and on-site engagement (the Round-2 failure modes), each with evidence and a severity.
- Suggested actions — what to change and how to fix each problem, prioritized. Suggestions may go beyond the detected problems: proactive improvements that would strengthen discoverability or engagement even where no explicit defect was found.

Recommend-only. No skill in the marketplace modifies a live website — the marketplace audits and reports. Suggested actions are recommendations, not changes applied to the site. No destructive, authenticated, or site-altering actions.

Sample audit report schema

Below is the minimum required shape for the entrypoint's audit report. It's a floor, not a ceiling — you may add fields, but every submission must include at least these.

```
{
  "site": "example.com",
  "audited_at": "2026-09-20T14:32:00Z",
  "summary": {
    "total_findings": 6,
    "critical": 1,
    "high": 2,
    "medium": 3
  },
  "findings": [
    {
      "id": "F-001",
      "title": "No JSON-LD structured data on product pages",
      "severity": "high",
      "evidence": "Crawled 12 product pages; 0/12 contain schema.org markup.",
      "suggested_action": {
        "summary": "Add Product/Offer JSON-LD to every product page.",
        "priority": "high"
      }
    }
  ]
}
```

Required fields per finding: id, title, severity, evidence, suggested_action. Required summary metadata: site, audited_at, and a counts-by-severity summary.

Example layout

(illustrative — any structure is fine as long as the rules below hold)

```
brand-ai-readiness-audit/      <- marketplace root (this is what you zip)
  marketplace.json            <- manifest: lists skills + the entrypoint
  skills/
    audit-orchestrator/      <- entrypoint: composes the others, emits the final
report
  SKILL.md
  scripts/
  references/
  crawl-render-audit/
  SKILL.md
  freshness-corroboration/
  SKILL.md
  engagement-audit/
  SKILL.md
README.md
```

Marketplace rules

agentskills.io defines the single-skill SKILL.md format but does not define a multi-skill marketplace format — the manifest below is this contest's own lightweight convention. Every skill folder inside your marketplace must still independently satisfy the official agentskills.io spec.

- Every skill folder listed in the marketplace must contain a valid SKILL.md per the agentskills.io spec (name, description, etc.). If you have Python/npm available, you can sanity-check with skills-ref validate ./skill-folder — not required, just a convenience.
- The marketplace root must include a top-level manifest, marketplace.json, listing every skill and marking exactly one as the entrypoint. The entrypoint is the skill we invoke; it's responsible for composing the outputs of any other skills in the marketplace into the single audit report described above.

```
{
  "name": "brand-ai-readiness-audit",
  "version": "1.0.0",
  "skills": [
    { "id": "audit-orchestrator", "path": "skills/audit-orchestrator", "entrypoint":
true },
    { "id": "crawl-render-audit", "path": "skills/crawl-render-audit" },
    { "id": "freshness-corroboration", "path": "skills/freshness-corroboration" },
    { "id": "engagement-audit", "path": "skills/engagement-audit" }
  ]
}
```

SKILL.md format (for each skill in the marketplace)

```
---
name: brand-ai-readiness-audit
description: Audit a website for AI-discoverability and on-site-engagement
  problems – crawlability, JS-render gaps, missing/invalid structured data,
  facts locked in non-text, stale or uncorroborated facts, entity ambiguity,
  weak on-site orientation / no context retention – and produce a report of
  findings plus prioritized, actionable fixes. Use when diagnosing why a brand
  is missing or misrepresented in AI assistants, or why visitors who arrive
  don't engage.
license: <your choice>
---
# Brand AI-Readiness Audit (discoverability + engagement)
## When to use
## Inputs (e.g. a URL / domain)
## Procedure (numbered, deterministic steps)
## Output (an audit report against a fixed schema: findings + suggested actions)
Keep each skill's SKILL.md lean — push detailed checklists to references/ and executable checks to scripts/
(progressive disclosure). Declare any allowed-tools. Narrowly-scoped skills (one concern each) compose more
cleanly under the entrypoint than one skill that tries to do everything.
```

2. What the marketplace must do

Below, "the skill" refers to whichever skill in your marketplace performs a given check — may be split across multiple skills or handled by a single one; either is fine as long as the entrypoint's final report covers everything required.

Detect — cover both halves of the Round-2 problem, reasoning from how these systems actually work (see the Round-2 appendix at the end of this document):

- Off-site discoverability — why the brand isn't found or cited by AI assistants.
- On-site engagement — why visitors who do arrive don't stay.

Working out the specific, concrete checks that surface each of these is part of the task — we're not providing a checklist here.

Report — for each problem found, record the evidence and severity, then give a suggested action: what to change and how, prioritized by impact.

Suggested actions may go beyond the problems found — proactive improvements that would strengthen AI discoverability or engagement even where no explicit defect was detected.

→ emit a single audit report (fixed schema): findings + suggested actions.

3. How submissions are evaluated

We evaluate the submitted marketplace itself — its skills' instructions, checks, logic, and how they're composed by the endpoint — not any single report it happens to produce. We look for:

- What problems it is built to identify — does the skill encode the right checks to surface the real causes of poor AI discoverability and engagement, with evidence and without false positives?
- How it addresses them — does the skill produce fixes that are correct, specific, mechanism-sound, and that actually resolve the problems it finds?
- How the marketplace is composed — if you split the work across multiple skills, does the decomposition reflect genuine separation of concerns and does the endpoint compose them cleanly, or is it padding? A single well-built skill is not penalized for not decomposing.
- What it does to improve AI discoverability & engagement beyond fixing specific defects — does it recommend actions that genuinely strengthen how the brand is found, cited, and experienced?
- Generalization — no example sites are provided during the task, so generalization is tested by construction.

4. Rubric

Criterion	Looking for
Detection accuracy	Correctly identifies the real problems (evidence-backed) across both discoverability and engagement, with few misses and few false positives
Suggested-action quality	Fixes are correctly targeted, mechanism-sound, and prioritized; beyond-problem suggestions are relevant and non-obvious
Output design	The marketplace's endpoint skill is built to emit a clear, structured, actionable report (evidence + severity + prioritized actions) a non-expert could act on
Skill-format & engineering hygiene	Each skill folder is agentskills.io compliant; the marketplace manifest is well-formed with exactly one endpoint; deterministic; safe
Marketplace composition	Decomposition into skills (if any) reflects genuine separation of concerns, with clean composition by the endpoint — not padding; a single well-built skill scores fully here too
Generalization	Works on unseen sites — no example sites are provided during the task, so generalization is tested by construction

5. Scope & guardrails

- Recommend-only — the marketplace audits and reports; no skill in it ever alters a live site, and everything runs read-only in a sandbox.
- No destructive, authenticated-area, or rate-abusing actions. Respect robots.txt.
- Each skill should be portable (the agentskills.io format is provider-neutral) and declare its tool needs; the marketplace manifest should be self-contained — no external service needed to resolve it.

Submission zip: ≤ 50 MB (no pre-trained model weights).

Audit runtime: < 5 minutes on a standard machine for a typical website.

6. Logistics

- Format: take-home — building and testing a skill doesn't fit a proctored hour.
- AI assistance is allowed in Round 3 — building agent tooling with an agent is the intended, realistic skill.
- Works on unseen sites — no example sites are provided during the task, so generalization is tested by construction.

Submission: a zip of the marketplace root directory (containing marketplace.json and every skill folder), with a short README.md at the root describing what each skill does and how the entrypoint composes them.

Appendix — Background Concepts (from Round 2)

This appendix explains the general ideas behind why AI assistants find, trust, and repeat some content and not others. It describes how these systems behave — not what to do about any particular site. Working out the checks and fixes is your job.

A. How search visibility works (the basics). Search engines, and increasingly AI assistants, discover content the same basic way: an automated program (a "crawler") visits a page, reads what's there, and stores it so it can be found later. For a page to be visible at all, three things have to succeed in order — the crawler has to be let in, it has to be able to read what's on the page, and it has to be able to pick out the specific fact someone is looking for. If any one of those steps fails, the page effectively doesn't exist for that system — plainly visible to a human, yet invisible to the machine.

B. How assistants like ChatGPT use sources. When you ask an assistant a question, it doesn't always answer purely from memory. Many assistants can go and look things up in the moment — searching, fetching a few pages, and building the answer from what those pages happen to say, sometimes showing links back to them. What ends up in the answer depends on what the assistant can find and use at that moment: the pages that get picked as sources tend to be the ones a machine could easily reach, easily read, and easily quote a clear fact from. If nothing about a brand is easy to reach, read, and quote, it simply won't make it into the answer at all.

C. How machines "read" a page. A page that looks complete to a person isn't always complete to a machine. Some content is only assembled after a page loads, or lives in a form a simple reader can't interpret — so a fact that's plainly visible on screen can be invisible to a program that isn't looking at it the same way. In general, the more explicitly and unambiguously a fact is stated in plain, readable text, the more likely a machine is to extract it correctly; the more it's implied, buried, or locked inside something non-textual, the more likely it's missed.

D. Why agreement across the web matters. Machines tend to treat a fact as more trustworthy when many independent places say the same thing. A claim that lives in only one spot is fragile; a claim repeated consistently across lots of unrelated sources is far more likely to be believed and repeated back. A related problem is mistaken identity: when several different things share a name, a system can mix them up unless there's something that clearly distinguishes one from the others. So how a brand is described across the wider web — not just on its own pages — shapes what assistants say about it.

E. Personalization and prior context. These assistants don't treat every user identically. They can draw on context they already hold about a person — earlier messages in the conversation, past questions, stated preferences, location, or other signals — and use it to tailor what they surface, so the answer is weighted toward what seems most relevant to that user. This is one more reason two people can ask the very same question and get different brands, framing, or emphasis in return: part of the answer depends not just on the content that exists out there, but on who's asking.

F. Why machines drop content from emails. Inboxes increasingly show short AI-generated summaries of long messages. Those summaries are built from the text of a message. When the real substance of an email isn't available as readable text — for example, when it's carried by something a summarizer can't read, or when the genuinely important lines are surrounded by low-value filler — the summary has little to work with, and the important part can simply disappear.